

第 2 章 形式语言基础

王 鑫

wangx AT tju.edu.cn

formal

天津大学 智能与计算学部

language



Colorless green ideas sleep furiously.

— Noam Chomsky

内容提要

- 1 问题的提出
- 2 形式文法与形式语言
- 3 乔姆斯基分类

内容提要

① 问题的提出

② 形式文法与形式语言

③ 乔姆斯基分类

例 (类 Pascal 语言的语句)

<语句> ::= <条件语句> | <当语句> | <复合语句> | <赋值语句>
<条件语句> ::= if <布尔表达式> then <语句> else <语句>
<当语句> ::= while <布尔表达式> do <语句>
<复合语句> ::= begin <语句表> end
<语句表> ::= <语句> | <语句> ; <语句表>
<赋值语句> ::= <变量> := <算术表达式>
<布尔表达式> ::= <算术表达式> <关系运算符> <算术表达式>
<关系运算符> ::= < | > | <= | >= | = | !=
<算术表达式> ::= <常量> | <变量> |
 (<算术表达式> <算术运算符> <算术表达式>)
<算数运算符> ::= + | - | * | /
<常量> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<变量> ::= a | b | c | d | e | f | g | h | i | j | k | l | m |
 n | o | p | q | r | s | t | u | v | w | x | y | z

BNF: 巴科斯-瑙尔范式

巴科斯-瑙尔范式 (Backus-Naur Form)

- $\langle \text{语句} \rangle$ 一种语法成分, 需要定义
- $::=$ “就是”
- $|$ “或者”



John Backus



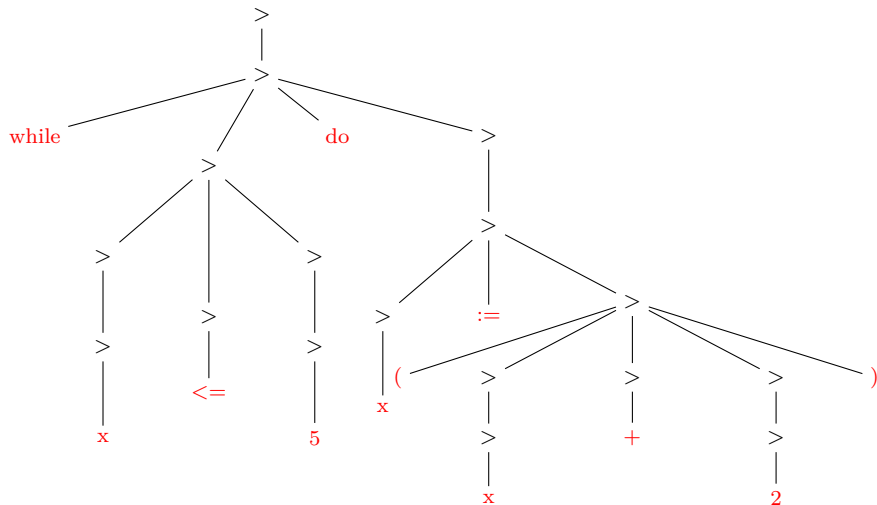
Peter Naur

递归定义

- $\langle \text{语句} \rangle$ 是 $\langle \text{语句表} \rangle$
- 在 $\langle \text{语句表} \rangle$ 之前加上 $\langle \text{语句} \rangle$; , 仍是 $\langle \text{语句表} \rangle$
- 除此之外, $\langle \text{语句表} \rangle$ 不再包含其他形式

例

```
while x <= 5 do x := (x + 2)
```



举例：简单英语句子的语法规则

例

<Sentence> → <Noun phrase><Verb phrase>

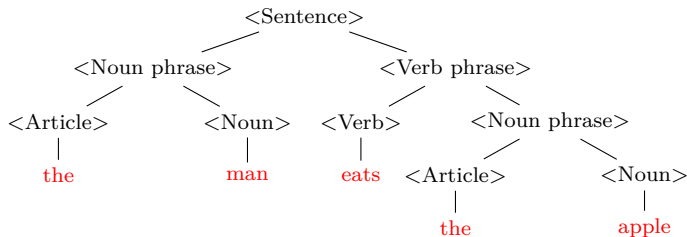
<Noun phrase> → <Article><Noun>

<Article> → the | a

<Noun> → apple | cat | man

<Verb phrase> → <Verb><Noun phrase> | <Verb>

<Verb> → eats | sings | runs



内容提要

- 1 问题的提出
- 2 形式文法与形式语言
- 3 乔姆斯基分类

定义

一个**文法** G 是一个四元组

$$G = (V, T, P, S)$$

其中

- ① V 是**变元**的有限集。
- ② T 是**终结符**的有限集。
- ③ P 是**产生式**的有限集，其中每个产生式都是 $\alpha \rightarrow \beta$ 的形式， $\alpha \in (V \cup T)^+$ ，且至少有一个 V 中的符号， $\beta \in (V \cup T)^*$ 。 α 称为产生式的**左部**， β 称为产生式的**右部**。
- ④ $S \in V$ ，称为文法 G 的**开始符号**。

约定

- ① **变元**: 大写拉丁字母 A, B, C, D, E 和 S , S 开始符号 (除非另作说明)。
- ② **终结符**: 小写拉丁字母 a, b, c, d, e , 数字。
- ③ **终结字符串**: 小写拉丁字母 u, v, w, x, y, z 等。
- ④ **变元和终结符共同组成的串**: 小写希腊字母 α, β, γ 等。

约定

$$\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$$

缩写为: $\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

例

写一个文法，只写出产生式集合便可

例

$$S \rightarrow 0A1 \mid 10$$

$$0A \rightarrow 00A1$$

$$A \rightarrow \varepsilon$$

推导

定义 (直接推导)

给定文法 $G = (V, T, P, S)$, 定义两个字符串之间的关系 $\Rightarrow$:
若 $\alpha = \alpha_1\alpha_2\alpha_3$, $\gamma = \alpha_1\beta\alpha_3$, 并且 $\alpha_2 \rightarrow \beta$ 是 P 中的一条产生式, 则有 $\alpha \Rightarrow \gamma$, 此时称由 α **直接推导** 出 γ

定义 (推导)

$\Rightarrow$ 是 $(V \cup T)^*$ 上的二元关系。根据关系闭包的定义, 可将 $\Rightarrow$ 扩充为 $\Rightarrow^*$, $\alpha \Rightarrow^* \gamma$ 称为由 α **推导** 出 γ 。

定义 (句型、句子)

若有 $S \Rightarrow^* \gamma$, 则称 γ 为 **句型**, 当 $\gamma \in T^*$ 时, 称 γ 为 **句子**。

例

文法

例

$$S \rightarrow 0A1 \mid 10$$

$$0A \rightarrow 00A1$$

$$A \rightarrow \varepsilon$$

有推导

例

$$S \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000111$$

$$S \overset{*}{\Rightarrow} 000111$$

其中哪些是句型？哪些是句子？

定义 (归约)

如果 $\alpha \Rightarrow \gamma$ 是由 α 到 γ 的推导, 则反过来称 γ **归约** 到 α , 记作 $\gamma \Leftarrow \alpha$

定义 (语言)

文法 $G = (V, T, P, S)$ 所产生的**语言**记作 $L(G)$, 定义为:

$$L(G) = \{w \mid S \xRightarrow{*} w \wedge w \in T^*\}$$

文法 G 产生的语言 $L(G)$, 就是由 G 中开始符号 S 推导出来的全体终结字符串的集合, 即**句子的集合**。

举例：由文法到语言

例

给定文法 G ，它有两个产生式：

$$S \rightarrow aSb$$

$$S \rightarrow ab$$

求该文法产生的语言 $L(G)$

$$L(G) = \{a^n b^n \mid n \geq 1\}$$

举例：由文法到语言

例

给定文法 G ，它的产生式为：

$$S \rightarrow aB \mid bA$$

$$A \rightarrow a \mid bAA \mid aS$$

$$B \rightarrow b \mid aBB \mid bS$$

求该文法产生的语言 $L(G)$

$L(G)$ 是由个数相等的 a 和 b （次序不限）组成的所有串的集合。

证明：板书

为了证明最终的结论，我们要证明以下三个互相关联的命题：

- ① $S \xRightarrow{*} w$ 当且仅当 w 中包含相等个数的 a 和 b
- ② $A \xRightarrow{*} w$ 当且仅当 w 中 a 的个数比 b 的个数多 1
- ③ $B \xRightarrow{*} w$ 当且仅当 w 中 b 的个数比 a 的个数多 1

举例：由文法到语言

为了证明最终的结论，我们要证明以下三个互相关联的命题：

- ① $S \xRightarrow{*} w$ 当且仅当 w 中包含相等个数的 a 和 b
- ② $A \xRightarrow{*} w$ 当且仅当 w 中 a 的个数比 b 的个数多 1
- ③ $B \xRightarrow{*} w$ 当且仅当 w 中 b 的个数比 a 的个数多 1

Proof.

用归纳法证明，对 $|w|$ 进行归纳。

归纳基础：

当 $|w| = 1$ 时，

对于命题 (1)，一方面，根据产生式，不可能有 $S \xRightarrow{*} w$

另一方面， w 中不可能包含相等个数的 a 和 b

即“当且仅当”的两个方面条件都是假的，故命题 (1) 成立。**为什么？**

对于命题 (2)(3)，只能有 $A \Rightarrow a$ 和 $B \Rightarrow b$ ，用其他产生式都将推导出长度大于 1 的串，故命题 (2)(3) 成立。□

- ① $S \xrightarrow{*} w$ 当且仅当 w 中包含相等个数的 a 和 b
- ② $A \xrightarrow{*} w$ 当且仅当 w 中 a 的个数比 b 的个数多 1
- ③ $B \xrightarrow{*} w$ 当且仅当 w 中 b 的个数比 a 的个数多 1

Proof.

用归纳法证明，对 $|w|$ 进行归纳。

归纳步骤：

假设当 $|w| \leq k-1$ 时，命题 (1)(2)(3) 成立。（归纳假设）

要证当 $|w| = k$ 时，命题 (1)(2)(3) 也成立。

先证命题 (1)

“仅当”：“ $S \xrightarrow{*} w$ ” $\implies$ “ w 中包含相等个数的 a 和 b ”

由 $S \xrightarrow{*} w$ ，第 1 步推导必然是 $S \Rightarrow aB$ 或 $S \Rightarrow bA$

- 若第 1 步推导是 $S \Rightarrow aB$ ；必然有 $w = aw_1$ 且 $B \xrightarrow{*} w_1$ ；
因为 $|w_1| = k-1$ ，根据归纳假设（命题 (3)）， w_1 中 b 的个数比 a 的个数多 1，因此， w 中包含相等个数的 a 和 b
- 若第 1 步推导是 $S \Rightarrow bA$ ；必然有 $w = bw_1$ 且 $A \xrightarrow{*} w_1$ ；
因为 $|w_1| = k-1$ ，根据归纳假设（命题 (2)）， w_1 中 a 的个数比 b 的个数多 1，因此， w 中包含相等个数的 a 和 b

- ① $S \xrightarrow{*} w$ 当且仅当 w 中包含相等个数的 a 和 b
- ② $A \xrightarrow{*} w$ 当且仅当 w 中 a 的个数比 b 的个数多 1
- ③ $B \xrightarrow{*} w$ 当且仅当 w 中 b 的个数比 a 的个数多 1

Proof.

用归纳法证明, 对 $|w|$ 进行归纳。

归纳步骤:

假设当 $|w| \leq k-1$ 时, 命题 (1)(2)(3) 成立。(归纳假设)

要证当 $|w| = k$ 时, 命题 (1)(2)(3) 也成立。

先证命题 (1)

“当”: “ w 中包含相等个数的 a 和 b ” $\implies$ “ $S \xrightarrow{*} w$ ”

 w 的第 1 个符号只有 a 或 b 两种情况 ($|w| = k$)

- 若 w 的第 1 个符号是 a ; 则 $w = aw_1$ 且 $|w_1| = k-1$;
因为 w_1 中包含 b 的个数比 a 的个数多 1, 根据归纳假设 (命题 (3)), 有 $B \xrightarrow{*} w_1$, 因此, 有 $S \Rightarrow aB \xrightarrow{*} aw_1 = w$, 即 $S \xrightarrow{*} w$
- 若 w 的第 1 个符号是 b ; 则 $w = bw_1$ 且 $|w_1| = k-1$;
因为 w_1 中包含 a 的个数比 b 的个数多 1, 根据归纳假设 (命题 (2)), 有 $A \xrightarrow{*} w_1$, 因此, 有 $S \Rightarrow bA \xrightarrow{*} bw_1 = w$, 即 $S \xrightarrow{*} w$

- ① $S \xrightarrow{*} w$ 当且仅当 w 中包含相等个数的 a 和 b
- ② $A \xrightarrow{*} w$ 当且仅当 w 中 a 的个数比 b 的个数多 1
- ③ $B \xrightarrow{*} w$ 当且仅当 w 中 b 的个数比 a 的个数多 1

Proof.

用归纳法证明，对 $|w|$ 进行归纳。

归纳步骤：

假设当 $|w| \leq k-1$ 时，命题 (1)(2)(3) 成立。（归纳假设）

要证当 $|w| = k$ 时，命题 (1)(2)(3) 也成立。

再证命题 (2)

“仅当”：“ $A \xrightarrow{*} w$ ” $\implies$ “ w 中 a 的个数比 b 的个数多 1”

由 $A \xrightarrow{*} w$ ，第 1 步推导必然是 $A \Rightarrow aS$ 或 $A \Rightarrow bAA$

- 若第 1 步推导是 $A \Rightarrow aS$ ；必然有 $w = aw_1$ 且 $S \xrightarrow{*} w_1$ ；
因为 $|w_1| = k-1$ ，根据归纳假设（命题 (1)）， w_1 中包含相等个数的 a 和 b ，因此， w 中 a 的个数比 b 的个数多 1
- 若第 1 步推导是 $A \Rightarrow bAA$ ；必然有 $w = bw_1w_2$ 且 $A \xrightarrow{*} w_1$ 和 $A \xrightarrow{*} w_2$ ；因为 $|w_1| < k-1$ 且 $|w_2| < k-1$ ，根据归纳假设（命题 (2)）， w_1 和 w_2 中 a 的个数比 b 的个数多 1，因此， w 中 a 的个数比 b 的个数多 1

- ① $S \xRightarrow{*} w$ 当且仅当 w 中包含相等个数的 a 和 b
- ② $A \xRightarrow{*} w$ 当且仅当 w 中 a 的个数比 b 的个数多 1
- ③ $B \xRightarrow{*} w$ 当且仅当 w 中 b 的个数比 a 的个数多 1

Proof.

用归纳法证明，对 $|w|$ 进行归纳。

归纳步骤：

假设当 $|w| \leq k-1$ 时，命题 (1)(2)(3) 成立。（归纳假设）

要证当 $|w| = k$ 时，命题 (1)(2)(3) 也成立。

再证命题 (2)

“当”： $\frac{\text{“}w \text{ 中 } a \text{ 的个数比 } b \text{ 的个数多 } 1\text{”} \implies \text{“}A \xRightarrow{*} w\text{”}}{}$

w 的第 1 个符号只有 a 或 b 两种情况 ($|w| = k$)

- 若 w 的第 1 个符号是 a ；则 $w = aw_1$ 且 $|w_1| = k-1$ ；
因为 w_1 中包含相等个数的 a 和 b ，根据归纳假设（命题 (1)），有 $S \xRightarrow{*} w_1$ ，因此，
有 $A \Rightarrow aS \xRightarrow{*} aw_1 = w$ ，即 $A \xRightarrow{*} w$
- 若 w 的第 1 个符号是 b ；则 $w = bw_1$ 且 $|w_1| = k-1$ ；
这时 w_1 中 a 的个数比 b 的个数多 2，可将 w_1 分为两部分，即 $w_1 = w_{11}w_{12}$ ，使
 w_{11} 和 w_{12} 中 a 的个数都比 b 的个数多 1
因为 $|w_{11}| < k-1$ 且 $|w_{12}| < k-1$ ，根据归纳假设（命题 (2)），有 $A \xRightarrow{*} w_{11}$ 和
 $A \xRightarrow{*} w_{12}$ ，因此，有 $A \Rightarrow bAA \xRightarrow{*} bw_{11}w_{12} = bw_1 = w$ ，即 $S \xRightarrow{*} w$

- ① $S \xrightarrow{*} w$ 当且仅当 w 中包含相等个数的 a 和 b
- ② $A \xrightarrow{*} w$ 当且仅当 w 中 a 的个数比 b 的个数多 1
- ③ $B \xrightarrow{*} w$ 当且仅当 w 中 b 的个数比 a 的个数多 1

Proof.

用归纳法证明，对 $|w|$ 进行归纳。

归纳步骤：

假设当 $|w| \leq k-1$ 时，命题 (1)(2)(3) 成立。(归纳假设)

要证当 $|w| = k$ 时，命题 (1)(2)(3) 也成立。

再证命题 (3)

与命题 (2) 的证明类似，留做作业 命题 (1)(2)(3) 成立。



举例：由语言到文法

例

给定语言 $L = \{a^n \mid n \geq 1\}$ ，构造产生它的文法。

$$S \rightarrow a$$

$$S \rightarrow aS$$

$$S \rightarrow a$$

$$S \rightarrow Sa$$

该例较为简单

举例：由语言到文法

例

给定语言 $L = \{ww^R \mid w \in \{a, b\}^+\}$, 构造产生它的文法。

- (1) $S \rightarrow aa$
- (2) $S \rightarrow bb$
- (3) $S \rightarrow aSa$
- (4) $S \rightarrow bSb$

$$S \Rightarrow aSa \Rightarrow aaaa$$

$$S \Rightarrow aSa \Rightarrow abba$$

$$S \Rightarrow bSb \Rightarrow baab$$

$$S \Rightarrow bSb \Rightarrow bbbb$$

举例：由语言到文法

更复杂

例

给定语言 $L = \{a^n b^n c^n \mid n \geq 1\}$ ，构造产生它的文法。

这样可以吗？

$$S \rightarrow abc$$

$$S \rightarrow aSbc$$

推导

$$S \rightarrow abc$$

$$S \xRightarrow{*} a^n(bc)^n$$

需要找出让 b 和 c 交换次序的方法？

举例：由语言到文法

例

给定语言 $L = \{a^n b^n c^n \mid n \geq 1\}$ ，构造产生它的文法。

如果有 $cb \rightarrow bc$ ，则问题就解决了。

可惜， $cb \rightarrow bc$ 不符合产生式的定义。为什么？

考虑 $cB \rightarrow Bc$

然后再设法将 B 换成 b

举例：由语言到文法

例

给定语言 $L = \{a^n b^n c^n \mid n \geq 1\}$ ，构造产生它的文法。

- (1) $S \rightarrow aBc$
- (2) $S \rightarrow aSBc$
- (3) $cB \rightarrow Bc$

看看 S 能推导出什么？

用产生式 (1)(2): $S \xRightarrow{*} a^n (Bc)^n$

用产生式 (3) 若干次，将所有的 B 移到 c 之前: $S \xRightarrow{*} a^n B^n c^n$

最后考虑如何将 B 换成 b ?

加一个产生式 $B \rightarrow b$ 可以吗?

举例：由语言到文法

- (1) $S \rightarrow aBc$
- (2) $S \rightarrow aSBc$
- (3) $cB \rightarrow Bc$

否

- 因为在推导过程中，不能限制何时才能使用 $B \rightarrow b$
- 如果在 B 还没有换到 c 之前就用了 $B \rightarrow b$ ，则结果仍然有 c 在 b 前的情况，且以后无法再换过去了。

例如， $S \Rightarrow aSBc \Rightarrow aaBcBc \xRightarrow{*} aabcbcb$

该文法产生了比要求更多的东西。

思考如何避免这一问题？

举例：由语言到文法

$$(1) S \rightarrow aBc$$

$$(2) S \rightarrow aSBc$$

$$(3) cB \rightarrow Bc$$

- 要限制 B 在 c 后面就被换成 b
- 必须保证当 B 前面是 a 时才开始进行 B 到 b 的替换

因此，需要产生式：

$$(4) aB \rightarrow ab$$

为了将所有 B 都换成 b ，还要有产生式：

$$(5) bB \rightarrow bb$$

举例：由语言到文法

例

给定语言 $L = \{a^n b^n c^n \mid n \geq 1\}$ ，构造产生它的文法。

产生该语言的完整的文法 G 是：

- (1) $S \rightarrow aBc$
- (2) $S \rightarrow aSBc$
- (3) $cB \rightarrow Bc$
- (4) $aB \rightarrow ab$
- (5) $bB \rightarrow bb$

证明：板书

举例：由语言到文法

例

给定语言 $L = \{a^n b^n c^n \mid n \geq 1\}$ ，构造产生它的文法。

产生该语言的完整的文法 G 是：

- (1) $S \rightarrow aBc$
- (2) $S \rightarrow aSBc$
- (3) $cB \rightarrow Bc$
- (4) $aB \rightarrow ab$
- (5) $bB \rightarrow bb$

Proof.

$$L(G) = \{a^n b^n c^n \mid n \geq 1\}$$



文法等价

定义 (文法等价)

对于两个不同的文法 $G_1 = (V_1, T_1, P_1, S_1)$, $G_2 = (V_2, T_2, P_2, S_2)$, 如果 $L(G_1) = L(G_2)$, 则称文法 G_1 与 G_2 **等价**。

举例：文法等价

例

构造产生全部十进制整数（每个整数之前可有若干个 0）的文法。

文法 G_1

$$N \rightarrow D \mid DN$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

文法 G_2

$$N \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

$$N \rightarrow 0N \mid 1N \mid 2N \mid 3N \mid 4N \mid 5N \mid 6N \mid 7N \mid 8N \mid 9N$$

内容提要

- 1 问题的提出
- 2 形式文法与形式语言
- 3 乔姆斯基分类**



(Speaking at a conference about humanity's prospects for survival in Amherst, MA,
April 17, 2017)

Noam Chomsky

Dec 7, 1928 (age 90)

语言学家、哲学家、认知科学家、政治评论家、社会活动家

MIT 语言学与哲学系教授

“现代语言学之父”

定义 (Chomsky hierarchy)

对于文法 $G = (V, T, P, S)$, 按以下标准分为 4 类:

- ① 若 P 中的产生式按照文法定义中给出的形式而不加额外的限制, 即每个产生式都是 $\alpha \rightarrow \beta$ 的形式, 其中, $\alpha \in (V \cup T)^+$, 且至少有一个 V 中的符号, $\beta \in (V \cup T)^*$, 则称 G 为 **0 型文法** (Type-0 Grammar) 或 **无限制文法** (Unrestricted Grammar), 简记为 UG

乔姆斯基分类

定义 (Chomsky hierarchy)

对于文法 $G = (V, T, P, S)$, 按以下标准分为 4 类:

- ② 若 P 中的每个产生式都具有如下形式:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

其中, $A \in V$, $\alpha, \beta \in (V \cup T)^*$, $\gamma \in (V \cup T)^+$

且产生式 $S \rightarrow \varepsilon$ 允许出现, 只要 S 不出现在任何产生式的右部, 则称 G 为 **1 型文法** (Type-1 Grammar) 或 **上下文有关文法** (Context-Sensitive Grammar), 简记为 CSG

对于 0 型文法, 可以有 $\alpha \rightarrow \beta$, $\beta = \varepsilon$
所以能产生空串。

乔姆斯基分类

定义 (Chomsky hierarchy)

对于文法 $G = (V, T, P, S)$, 按以下标准分为 4 类:

- ③ 若 P 中的每个产生式都具有如下形式:

$$A \rightarrow \beta$$

其中, $\beta \in (V \cup T)^*$, $A \in V$

则称 G 为 **2 型文法** (Type-2 Grammar) 或 **上下文无关文法** (Context-Free Grammar),
简记为 CFG

所有的程序设计语言都是

乔姆斯基分类

定义 (Chomsky hierarchy)

对于文法 $G = (V, T, P, S)$, 按以下标准分为 4 类:

- ① 若 P 中的每个产生式都具有如下形式:

$$A \rightarrow a$$

$$A \rightarrow aB$$

其中, $a \in T \cup \{\varepsilon\}$, $A, B \in V$

则称 G 为 **3 型文法** (Type-3 Grammar) 或 **正则文法** (Regular Grammar),

简记为 RG

乔姆斯基分类：语言

定义 (语言分类)

- ① 由无限制文法产生的语言称为**递归可枚举语言**，简记为 REL (Recursively Enumerable Language)
- ② 由上下文有关文法产生的语言称为**上下文有关语言**，简记为 CSL (Context-Sensitive Language)
- ③ 由上下文无关文法产生的语言称为**上下文无关语言**，简记为 CFL (Context-Free Language)
- ④ 由正则文法产生的语言称为**正则语言**，简记为 RL (Regular Language)

0 型文法：举例

显示 0 型文法的“威力”，迄今为止最复杂的例子

例

$$(1) S \rightarrow ACaB$$

$$(2) Ca \rightarrow aaC$$

$$(3) CB \rightarrow DB$$

$$(4) CB \rightarrow E$$

$$(5) aD \rightarrow Da$$

$$(6) AD \rightarrow AC$$

$$(7) aE \rightarrow Ea$$

$$(8) AE \rightarrow \varepsilon$$

这是一个“真正的”0 型文法。为什么？

0 型文法：举例

例

$$(1) S \rightarrow ACaB$$

$$(2) Ca \rightarrow aaC$$

$$(3) CB \rightarrow DB$$

$$(4) CB \rightarrow E$$

$$(5) aD \rightarrow Da$$

$$(6) AD \rightarrow AC$$

$$(7) aE \rightarrow Ea$$

$$(8) AE \rightarrow \varepsilon$$

要证明： $L(G) = \{a^{2^k} \mid k \geq 1\}$

0 型文法：举例

要证明： $L(G) = \{a^{2^k} \mid k \geq 1\}$

$$\{a^{2^k} \mid k \geq 1\} = \{aa, aaaa, aaaaaaaaa, \dots\}$$

- 该语言中包含的串都由 2^k ($k = 1, 2, 3, \dots$) 个 a 组成
- 产生它的文法必须有“计数”的功能
- 怎样才能做到这一点呢？

0 型文法：举例

例

(1) $S \rightarrow ACaB$ A 为左边界, B 为右边界, C 相当于“光标”

(2) $Ca \rightarrow aaC$

当光标从左到右越过一个 a 时, 就变成两个 a

$\therefore C$ 从左边界到右边界走一趟, a 个数增加一倍

(3) $CB \rightarrow DB$

(4) $CB \rightarrow E$

(5) $aD \rightarrow Da$

(6) $AD \rightarrow AC$

(7) $aE \rightarrow Ea$

(8) $AE \rightarrow \varepsilon$

0 型文法：举例

例

$$(1) S \rightarrow ACaB$$

$$(2) Ca \rightarrow aaC$$

光标 C 走到右边界 B 时，两边界间 a 个数一定是 2^k

此时有两种选择

$$(3) CB \rightarrow DB$$

$$(4) CB \rightarrow E$$

$$(5) aD \rightarrow Da$$

$$(6) AD \rightarrow AC$$

$$(7) aE \rightarrow Ea$$

$$(8) AE \rightarrow \varepsilon$$

0 型文法：举例

例

$$(1) S \rightarrow ACaB$$

$$(2) Ca \rightarrow aaC$$

选择一：就此结束。将边界之间所有 a 作为文法产生的句子

选择二：继续推导。使之产生更多的 a

$$(3) CB \rightarrow DB$$

$$(4) CB \rightarrow E$$

$$(5) aD \rightarrow Da$$

$$(6) AD \rightarrow AC$$

$$(7) aE \rightarrow Ea$$

$$(8) AE \rightarrow \varepsilon$$

0 型文法：举例

例

$$(1) S \rightarrow ACaB$$

$$(2) Ca \rightarrow aaC$$

选择一：就此结束。将边界之间所有 a 作为文法产生的句子

$$(3) CB \rightarrow DB$$

$$(4) CB \rightarrow E$$

将 CB 改为 E ，表示推导即将结束

$$(5) aD \rightarrow Da$$

$$(6) AD \rightarrow AC$$

$$(7) aE \rightarrow Ea$$

$$(8) AE \rightarrow \varepsilon$$

(7)(8) 处理善后事宜 $S \xRightarrow{*} Aaaaaa \cdots aaaE$ ，其中 a 为 2^k 个

0 型文法：举例

例

$$(1) S \rightarrow ACaB$$

$$(2) Ca \rightarrow aaC$$

选择二：继续推导。使之产生更多的 a

$$(3) CB \rightarrow DB$$

将 C 改为 D ， D 由右向左走

$$(4) CB \rightarrow E$$

$$(5) aD \rightarrow Da$$

D 由右向左走，遇到 a 交换位置

$$(6) AD \rightarrow AC$$

直到遇到左边界 A ，将 D 再改为 C ，这就构成了一次循环

0 型文法：举例

例

$$(1) S \rightarrow ACaB$$

$$(2) Ca \rightarrow aaC$$

$$(3) CB \rightarrow DB$$

$$(4) CB \rightarrow E$$

$$(5) aD \rightarrow Da$$

$$(6) AD \rightarrow AC$$

$$(7) aE \rightarrow Ea$$

$$(8) AE \rightarrow \varepsilon$$

要证明： $L(G) = \{a^{2^k} \mid k \geq 1\}$

严格证明：对推导步数用归纳法

0 型文法：举例

$$L(G) = \{a^{2^k} \mid k \geq 1\}$$

该语言不是 CFL。

也就是说，用上下文无关文法不能产生这个语言。

为什么？

以后讲

既然 2 型文法不能产生它，1 型文法如何呢？

答：1 型文法虽可以产生，但比前面的 0 型文法要复杂得多。

1 型文法：举例

构造一个 1 型文法，使其能产生语言 $\{a^{2^k} \mid k \geq 1\}$

$$(1) S \rightarrow [ACaB]$$

$$(2) [ACaB] \rightarrow [Aa][aCB]$$

$$[Ca]a \rightarrow aa[Ca]$$

$$[Ca][aB] \rightarrow aa[CaB]$$

$$[ACa]a \rightarrow [Aa]a[Ca]$$

$$[ACa][aB] \rightarrow [Aa]a[CaB]$$

$$[CaB] \rightarrow a[aCB]$$

$$(3) [aCB] \rightarrow [aDB]$$

$$(4) [aCB] \rightarrow [aE]$$

$$(5) a[Da] \rightarrow [Da]a$$

$$[aDB] \rightarrow [DaB]$$

$$[Aa][Da] \rightarrow [ADa]a$$

$$a[DaB] \rightarrow [Da][aB]$$

$$[Aa][DaB] \rightarrow [ADa][aB]$$

$$(6) [ADa] \rightarrow [ACa]$$

$$(7) [aE] \rightarrow [Ea]$$

$$a[Ea] \rightarrow [Ea]a$$

$$(8) [Aa][Ea] \rightarrow aa$$

1 型文法与 2 型文法

下面讨论

- 为什么 1 型文法叫上下文有关文法?
- 为什么 2 型文法叫上下文无关文法?

要回答这个问题, 先看 1^0 型文法的定义

1 型文法：举例

给出文法 G 如下：

例

$$S \rightarrow 0A1 \mid 1A0$$

$$0A1 \rightarrow 0a1$$

$$1A0 \rightarrow 1b0$$

根据定义， G 为 1 型文法

$$L(G) = \{0a1, 1b0\}$$

乍看：不就是将变元 A 换成 a 和 b 吗？

为什么不直接写出产生式 $A \rightarrow a$ 和 $A \rightarrow b$ 呢？

1 型文法：举例

文法 G' :

例

$$S \rightarrow 0A1 \mid 1A0$$

$$A \rightarrow a$$

$$A \rightarrow b$$

$$L(G') = \{0a1, 0b1, 1a0, 1b0\}$$

- G' 是 2 型文法，上下文无关文法
- 变元 A 无论出现在什么地方，都可以独立地换成 a 或 b

1 型文法：举例

给出文法 G 如下：

例

$$S \rightarrow 0A1 \mid 1A0$$

$$0A1 \rightarrow 0a1$$

$$1A0 \rightarrow 1b0$$

而 G 则不同

- A 只有在左邻为 0，右邻为 1 的环境下，才能换成 a
- A 只有在左邻为 1，右邻为 0 的环境下，才能换成 b

一个变元究竟能换成什么，是由它所处的语言环境决定的，这个语言环境就是“上下文”。

1 型文法：举例

在 $\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$ 中
 α_1, α_2 就是变元 A 的上下文
 A 只有在这样的上下文中才能换成 β

2 型文法中的变元则没有语言环境的限制
随时可以用它的产生式的右部替换
这就是“上下文无关”的文法了

本章总结

- 巴科斯-瑙尔范式
- 形式文法与形式语言
 - 文法
 - 推导
 - 句型、句子
 - 语言
- 乔姆斯基分类
 - 0 型文法 – 短语结构文法
 - 1 型文法 – 上下文有关文法
 - 2 型文法 – 上下文无关文法
 - 3 型文法 – 正则文法

